

Capstone Project: Combining Predictive Techniques

Instructor: Dr. Akash Gupta

Table of contents

1	Project Overview	2
1.1	Key Dates	2
1.2	Dataset	2
1.3	Rules of Engagement	3
2	Required Workflow	4
2.1	1. Problem statement	4
2.2	2. Exploratory Data Analysis	4
2.3	3. Data cleaning	4
2.4	4. Feature engineering	5
2.5	5. Train/test split	5
2.6	6. Model training	5
2.7	7. Evaluation	6
2.8	8. Recommended model	6
2.9	9. Interpretation	6
3	Deliverables	7
3.1	Deliverable 1 — Jupyter Notebook	7
3.2	Deliverable 2 — Business Memo	7
3.3	Deliverable 3 — Concept Reflection	8
3.4	Deliverable 4 — In-Person Walkthrough (May 4)	8
4	Rubric (100 points)	9
5	Getting Started	10
5.1	Folder structure	10
5.2	Claude Code setup	10
5.3	First-hour suggested start	10
5.4	A note on using Claude Code well	10

1 Project Overview

This capstone project asks you to take one real business problem from a real company and attack it with **multiple predictive techniques**, then recommend a single production model based on your analysis. You will work in a **team of 2**. The project is designed around building conceptual understanding — the rubric rewards **judgment and interpretation**, not lines of code.

The Business Problem

Company: Lending Club — a publicly-traded peer-to-peer lending platform.

Scenario: Lending Club assigns each loan application a letter **grade** (A through G) based on its internal risk model. Grades A, B, C are considered prime borrowers; grades D, E, F, G are considered higher-risk borrowers who pay higher interest.

A new product team at Lending Club wants a second-opinion model: given only a borrower's application characteristics (income, credit history, employment, loan purpose, etc.), can we predict whether Lending Club's internal model will classify them as higher-risk (D–G) versus prime (A–C)?

Your job: Build and compare **four predictive models**, recommend one for production, and defend the choice to a non-technical risk-management director.

1.1 Key Dates

Deadlines

- **May 4** — 10-minute in-person team walkthroughs in class. Your grade on this component will reflect how well your team explains the code and modeling choices.
- **May 4 at 5:00 PM** — All files (notebook, memo, concept reflection) due on Canvas.

Each student uploads the **same team files individually** to Canvas.

1.2 Dataset

Source: OpenIntro loans_full_schema (public release of Lending Club loan-level data). See https://www.openintro.org/data/index.php?data=loans_full_schema for the data dictionary.

File: data/lending_club_raw.csv (10,000 rows, 55 columns). The file is provided **raw and uncleaned**. Cleaning is part of the work.

Target variable you will engineer:

Target definition

risky_grade = 1 if grade is in {D, E, F, G}; risky_grade = 0 if grade is in {A, B, C}.
Approximately 18.5% of loans in this dataset are risky-grade. This class imbalance is realistic for credit-risk problems — you will need to think carefully about evaluation metrics and threshold choice.

1.3 Rules of Engagement

Rules — Read Carefully

1. You must be able to **explain every line of code** in your notebook. The instructor may ask either team member to verbally walk through any cell during the May 4 walkthrough.
2. Use `random_state=42` on all splits and models so your numbers are reproducible.
3. Claude Code is allowed and encouraged as a pair programmer. However, **your team owns the submission**. Memos and reflections that your team cannot explain on request will lose credit.
4. Evaluation is on **outcome**, not process. You are graded on the quality of your final submission.
5. Both team members submit the same files to Canvas individually by the May 4 deadline.

2 Required Workflow

Your Jupyter notebook must move through the following workflow **in order**. Each section corresponds to a required part of the rubric.

2.1 1. Problem statement

Restate the business problem and target variable in 3–5 sentences at the top of the notebook.

2.2 2. Exploratory Data Analysis

Produce at least **three visualizations** that build your understanding of the data before modeling. Good choices include:

- Distribution of the `risky_grade` target
- A missing-value heatmap or bar chart
- A plot showing how one numeric feature (e.g., `annual_income`, `debt_to_income`) relates to `risky_grade`

Each figure should have a short caption describing what you see.

2.3 3. Data cleaning

Data-leakage columns — you must remove all of these

The following columns are either **the target itself in disguise** or are only populated **after the loan has already been issued**. You must drop every one of them before training any model. In your notebook, include a markdown cell naming each column below and stating in one sentence why it is a leakage feature.

Drop these columns:

- `grade` — the target is derived directly from this column.
- `sub_grade` — finer version of `grade`; same problem.
- `interest_rate` — set by Lending Club based on `grade`, so it encodes the target.
- `installment` — derived from `loan_amount`, `term`, and `interest_rate`; inherits the leakage.
- `loan_status` — populated after loan origination, not available at application time.
- `initial_listing_status` — only known after Lending Club approves and lists the loan.
- `disbursement_method` — only known after approval.
- `balance` — post-origination balance; unknown at application time.
- `paid_total`, `paid_principal`, `paid_interest`, `paid_late_fees` — all post-origination outcomes.

Your cleaning section must also address:

- **Missing values** — for each column with more than ~5% missing, state your strategy (drop, impute, indicator) and why
- **String-to-numeric conversions** — several columns look numeric but are stored as text. Identify them and convert.
- **Categorical features** — decide on one-hot vs. ordinal encoding and justify
- **Joint-application columns** — three columns (`annual_income_joint`, `debt_to_income_joint`, `verification_income_joint`) are populated only for joint applications. Decide how to handle them.

Each cleaning decision must have a **markdown cell** immediately before the code that explains *what* and *why*.

2.4 4. Feature engineering

Create at least **three new features** from the raw columns. Each new feature needs a one-line rationale. Example ideas (you are not required to use these specific ones):

- `loan_to_income = loan_amount / annual_income`
- `credit_history_years = current_year - earliest_credit_line`
- `is_joint_application` (1 if joint application, 0 otherwise)
- `has_prior_delinquency` (1 if `months_since_last_delinq` is present, 0 otherwise)
- `issue_year` OR `issue_month_num` extracted from `issue_month`

2.5 5. Train/test split

Do an **80/20 train/test split** with `random_state=42`. The split must happen **before** any scaling or imputation is fit.

Why split first?

Scaling (or imputing) the full dataset before splitting lets test-set statistics leak into the training pipeline. Your test score would be inflated. Always: **split first, fit preprocessing on training data only, transform the test set using the training statistics**.

2.6 6. Model training

Train the following four models on the **same pipeline** (same preprocessing, same train split, same random state):

1. **Logistic Regression** with L2 regularization (try `C` in `[0.01, 0.1, 1, 10]`)
2. **Decision Tree**, pre-pruned with `max_depth` and `min_samples_leaf`
3. **Neural Network** — scikit-learn `MLPClassifier`, one hidden layer
4. **One ensemble** — either Random Forest **or** Gradient Boosting

All four models must see identical features. You may use a scikit-learn Pipeline with a `ColumnTransformer` to keep the code clean.

2.7 7. Evaluation

Produce a **single comparison table** with five columns of metrics on the held-out test set for all four models:

Model	Accuracy	Precision	Recall	F1	ROC-AUC
-------	----------	-----------	--------	----	---------

Also produce **one overlaid ROC curve** showing all four models on the same axes.

2.8 8. Recommended model

Pick one model and defend the choice. Display its **confusion matrix** at the chosen threshold.

2.9 9. Interpretation

Produce a ranked list of the **top 5 most predictive features** for your recommended model. For tree/ensemble models, use `feature_importances_` or permutation importance. For logistic regression, use scaled coefficient magnitude.

3 Deliverables

Each team produces **one shared set of deliverables**. Each student then uploads the **same files individually** to Canvas by **May 4 at 5:00 PM**.

Use [lastname1]_[lastname2] in every filename with both team members' last names in **alphabetical order**. Example: a team of Rivera and Chen becomes team_chen_rivera_*.

3.1 Deliverable 1 — Jupyter Notebook

Deliverable 1 — Notebook

Filename: team_[lastname1]_[lastname2]_capstone_notebook.ipynb

Format: Jupyter notebook with **all cells pre-run** so outputs are visible without re-execution. Also submit an HTML export (.html) alongside.

Must include all nine sections listed under "Required Workflow" above, in that order.

3.2 Deliverable 2 — Business Memo

Deliverable 2 — Business Memo

Filename: team_[lastname1]_[lastname2]_memo.pdf

Format: PDF. You may prepare the memo in Word, Google Docs, or any other tool — only the final PDF is submitted.

Length: 3 pages maximum (enforced).

Audience: Lending Club's head of risk management (**non-technical**). No Greek letters or matrix notation in the main text.

Required sections:

1. **Problem and stakes** (~¼ page) — what you are predicting and why it matters to the business.
2. **Recommendation** (~½ page) — which of the four models to deploy and a plain-English reason.
3. **Top 5 drivers** (~1 page) — the five most predictive features with a one-sentence business interpretation for each.
4. **Tradeoffs and threshold** (~½ page) — discussion of false-positive vs false-negative cost and how you chose the classification threshold.
5. **Operational recommendation** (~¼ page) — one concrete action the company should take, plus one deployment caveat.

3.3 Deliverable 3 — Concept Reflection

Deliverable 3 — Concept Reflection

Filename: team_[lastname1]_[lastname2]_concept_reflection.pdf

Format: PDF. Prepare in any tool you like; only the final PDF is submitted.

Length: 1 page maximum.

Content: 2–3-sentence answers to all five questions below.

The five concept-reflection questions:

1. You dropped at least one column as a data-leakage feature. Name the column and explain, in two sentences, why keeping it would have produced a great-looking but useless model.
2. Your neural network did **not** dramatically beat logistic regression on this dataset. What does that tell you about when neural networks are worth the added complexity?
3. If the cost of funding a bad loan is roughly **10 times** the cost of turning away a good borrower, how should the classification threshold shift, and why?
4. Why did your pipeline scale the features **after** the train/test split, not before? What specifically would have gone wrong if you scaled first?
5. If you had ordinal-encoded the Lending Club loan `grade` (A–G) as 1–7 instead of one-hot, what specific modeling problem could that create for logistic regression — and does it matter for decision trees?

3.4 Deliverable 4 — In-Person Walkthrough (May 4)

Deliverable 4 — Walkthrough

Date: May 4 in class.

Format: 5-minute team presentation, screen-sharing your notebook and memo. **No slide deck required.**

Cover: the problem, cleaning highlights, recommended model, top features, one question for the class.

Followed by: 2–3 minutes of questions from the instructor. Either team member may be asked to explain any part of the notebook or memo.

Grading: your grade on this component will reflect how clearly your team explains the code and modeling choices. Nothing to upload separately — the walkthrough is assessed during class.

4 Rubric (100 points)

Rubric

- **Notebook — cleaning & feature engineering: 25 points**
Thoughtful cleaning with reasoning, ≥ 3 engineered features with rationale, leakage handled correctly.
- **Notebook — modeling & evaluation: 25 points**
All four models trained on identical pipelines, clean metric table, ROC overlay, no data leakage.
- **Business memo: 25 points**
Plain-English, stakeholder-appropriate, recommendation defended, within page limit.
- **Concept reflection: 15 points**
Reasoning quality, not just surface-correct answers.
- **In-person walkthrough: 10 points**
Clarity of explanation; handles live Q&A.

Technical outcome (cleaning + modeling) = **50%**. Communication and reasoning about the outcome (memo + reflection + walkthrough) = **50%**.

5 Getting Started

5.1 Folder structure

Your working copy of this capstone folder contains:

- `topic_capstone_combining_predictive_techniques_instructions.pdf` (this document)
- `topic_capstone_combining_predictive_techniques_rubric.pdf` (grading rubric)
- `capstone_starter_notebook.ipynb` (skeleton to copy and rename)
- `data/lending_club_raw.csv` (raw data)
- `README.md` (quick-start checklist)

5.2 Claude Code setup

Before starting, make sure Claude Code is installed and working. See the existing guides in `/setup_guides/`:

- `vs_code_claude_code_installation.pdf` — one-time installation steps
- `using_claude_code.pdf` — prompt patterns, plan mode, CLAUDE.md basics

5.3 First-hour suggested start

1. Both teammates read this PDF end to end.
2. Open `capstone_starter_notebook.ipynb` and save a copy with your team name.
3. Load the CSV and run the provided inspection cells to get a feel for the data.
4. As a team, decide which columns you suspect are leakage and write them into a markdown cell before touching them.
5. Draft your cleaning plan as bullet points in a markdown cell — then start implementing, one column group at a time.

5.4 A note on using Claude Code well

Claude Code is most useful for scaffolding, debugging, and explaining unfamiliar code. It is least useful (and most dangerous) when you paste its output without reading. Two habits that separate strong teams from weak teams:

- **Before accepting code:** read it, decide if each line makes sense, and make at least one edit of your own.
- **Before accepting an explanation:** ask Claude for the two or three assumptions it is making. If any of those assumptions do not hold, do not use the answer.

Good luck.